Chapter 5

Considering Other Task Types

IN THIS CHAPTER

- **Moving forward with text processing**
- **Performing sentiment analysis**
- **Working through a time series**

Book 4, Chapter 4 introduces you to the topic of Natural Language Processing (NLP), where you consider how a computer can process text despite not understanding it. The chapter points out that programming a computer to process human language is a daunting task, which is only recently possible using Natural Language Processing (NLP), deep learning Recurrent Neural Networks (RNNs), and word embeddings. This chapter takes you further by looking more closely at tokenization, the bag-of-words approach to analysis, and sentiment analysis. These approaches rely on building a model using Keras and employing deep learning techniques.

REMEMBER You don't have to type the source code for this chapter manually. In fact, using the downloadable source is a lot easier. The source code for this chapter appears in the `DSPD_0505_Other_Tasks.ipynb` source code file for Python and the `DSPD_R_0505_Other_Tasks.ipynb`

source code file for R. See the Introduction for details on how to find these source files.

Processing Language in Texts

Even though Book 4, Chapter 4 shows how to turn text (no matter what source you use) into data that a computer can analyze, it never really gets into the issue of language, which is a lot more than simply text. Language includes nuanced terms and hidden meanings that express more than the text would say on its own. Obviously, a computer can't understand things like sentiment, but you can make it appear that it does to a degree by performing certain types of deep learning analysis, which is where the following sections lead you. Of course, no matter how much analysis you perform, a computer can't ever discover the true meaning of sentences said with differing vocal inflections or possessing hidden meanings known only to the people engaging in the conversation.

Considering the processing methodologies

As a simplification, you can view language as a sequence of words made of letters (as well as punctuation marks, symbols, emoticons, and so on). Deep learning processes language best by using layers of RNNs, such as Long Short-Term Memory (LSTM) or Gated Recurrent Units (GRU). (Chapter 11 of *Deep Learning For Dummies*, by John Paul Mueller and Luca Massaron [Wiley], explains the use of LTSM and GRU.) However, knowing to use RNNs doesn't tell you how to use sequences as inputs; you need to determine the kind of sequences. In fact, deep learning networks accept only numeric input values. Computers encode letter sequences that you understand into numbers according to a protocol, such as Unicode Transformation Format-8 bit (UTF-8). UTF-8 is the most widely used encoding. (You can read the primer about encodings at `https://www.alexreisner.com/code/character-encoding`.)

REMEMBER Deep learning can also process textual data using Convolutional Neural Networks (CNNs) instead of RNNs by representing sequences as matrices (similar to image processing). Keras supports CNN layers, such as the `Conv1D` (`https://keras.io/layers/convolutional/`), which can operate on ordered features in time — that is, sequences of words or other signals. The 1D convolution output is usually followed by a `MaxPooling1D` layer that summarizes the outputs. CNNs applied to sequences find a limit in their insensitivity to the global order of the sequence. (They tend to spot local patterns.) For this reason, they're best used in sequence processing in combination with RNNs, not as their replacement.

Natural Language Processing (NLP) consists of a series of procedures that improve the processing of words and phrases for statistical analysis, machine learning algorithms, and deep learning. NLP owes its roots to computational linguistics that powered AI rule-based systems, such as expert systems, which made decisions based on a computer translation of human knowledge, experience, and way of thinking. NLP digested textual information, which is unstructured, into more structured data so that expert systems could easily manipulate and evaluate it.

Deep learning has taken the upper hand today, and expert systems are limited to specific applications in which interpretability and control of decision processes are paramount (for instance, in medical applications and driving-behavior decision systems in some self-driving cars). Yet, the NLP pipeline is still quite relevant for many deep learning applications.

Defining understanding as tokenization

In an NLP pipeline, the first step is to obtain raw text. Usually you store it in memory or access it from disk. When the data is too large to fit in memory, you maintain a pointer to it on disk (such as the directory name and the filename). In the following example, you use three documents (represented by string variables) stored in a list (in computational linguistics, the document container is the *corpus*):

```
import numpy as np

texts = ["My dog gets along with cats",
         "That cat is vicious",
         "My dog is happy when it is lunch"]
```

After obtaining the text, you process it. As you process each phrase, you extract the relevant features from the text (you usually create a *bag-of-words* matrix) and pass everything to a learning model, such as a deep learning algorithm. During text processing, you can use different transformations to manipulate the text (with tokenization being the only mandatory transformation):

- **Tokenization (mandatory):** Split a sentence into individual words.
- **Cleaning:** Remove nontextual elements such as punctuation and numbers.
- **Lemmatization:** Transform a word into its dictionary form (the lemma). It's an alternative to stemming, but it's more complex because you don't use an algorithm. Instead, you use a dictionary to convert every word into its lemma.
- **N-grams:** Associate every word with a certain number (the *n* in n-gram), of following words and treat them as a unique set. Usually, *bi-grams* (a series of two adjacent elements or tokens) and *tri-grams* (a series of three adjacent elements or tokens) work the best for analysis purposes.
- **Normalization:** Remove capitalization.
- **Pos-tagging:** Tag every word in a phrase with its grammatical role in the sentence (such as tagging a word as a verb or as a noun). You can read more

about Parts of Speech (POS) tagging at `https://medium.com/analytics-vidhya/pos-tagging-using-conditional-random-fields-92077e5eaa31`.

- **Stemming:** Reduce a word to its stem (which is the word form before adding inflectional affixes, as you can read here: `https://www.thoughtco.com/stem-word-forms-1692141`). An algorithm, called a stemmer, can do this based on a series of rules.
- **Stop word removal:** Remove common, uninformative words that don't add meaning to the sentence, such as the articles *the* and *a*. Removing negations such as *not* could be detrimental if you want to guess the sentiment.

To achieve these transformations, you may need a specialized Python package such as NLTK (`http://www.nltk.org/api/nltk.html`) or Scikit-learn (see the tutorial at `https://scikit-learn.org/stable/tutorial/text_analytics/working_with_text_data.html`). When working with deep learning and a large number of examples, you need only basic transformations: normalization, cleaning, and tokenization. The deep learning layers can determine what information to extract and process. When working with few examples, you do need to provide as much NLP processing as possible to help the deep learning network determine what to do in spite of the little guidance provided by the few examples.

TIP Keras offers a function, `keras.preprocessing.text.Tokenizer`, that normalizes (using the lower parameter set to True), cleans (the filters parameter contains a string of the characters to remove, usually these: '!"#$%&()*+,-./:;<=>?@[\]^_`{|}~ '), and tokenizes.

Putting all the documents into a bag

After processing the text, you have to extract the relevant features, which means transforming the remaining text into numeric information for the neural network to process. This is commonly done using the bag-of-words approach, which is obtained by frequency encoding

or binary encoding the text. This process equates to transforming each word into a matrix column as wide as the number of words you need to represent. The following example shows how to achieve this process and what it implies.

Obtaining the vocabulary size

The example uses the `texts` list instantiated earlier in the chapter. As a first step, you prepare a basic normalization and tokenization using a few Python commands to determine the word vocabulary size for processing:

```
unique_words = set(word.lower() for phrase in texts for
                   word in phrase.split(" "))
print(f"There are {len(unique_words)} unique words")
```

When you run this code using the `texts` list defined in the previous section, you see this output:

```
There are 14 unique words
```

Processing the text

You now proceed to load the `Tokenizer` function from Keras and set it to process the text by providing the expected vocabulary size:

```
from keras.preprocessing.text import Tokenizer
vocabulary_size = len(unique_words) + 1
tokenizer = Tokenizer(num_words=vocabulary_size)
```

TIP Using a `vocabulary_size` that's too small may exclude important words from the learning process. One that's too large may uselessly consume computer memory. You need to provide `Tokenizer` with a correct estimate of the number of distinct words contained in the list of texts. You also always add 1 to the `vocabulary_size` to provide an extra word for the start of a phrase (a term that helps the deep learning network). At this point, `Tokenizer` maps the words present in the texts to indexes, which are numeric values representing the words in text:

```
tokenizer.fit_on_texts(texts)
```

```
print(tokenizer.index_word)
```

The resulting indexes are as follows:

```
{1: 'is', 2: 'my', 3: 'dog', 4: 'gets', 5: 'along',
 6: 'with', 7: 'cats', 8: 'that', 9: 'cat', 10: 'vicious',
 11: 'happy', 12: 'when', 13: 'it', 14: 'lunch'}
```

The indexes represent the column number that houses the word information:

```
  print(tokenizer.texts_to_matrix(texts))
```

Here's the resulting matrix:

```
[[0. 0. 1. 1. 1. 1. 1. 1. 0. 0. 0. 0. 0. 0. 0.]
 [0. 1. 0. 0. 0. 0. 0. 0. 1. 1. 1. 0. 0. 0. 0.]
 [0. 1. 1. 1. 0. 0. 0. 0. 0. 0. 0. 1. 1. 1. 1.]]
```

The matrix consists of 15 columns (14 words plus the start of phrase pointer) and three rows, representing the three processed texts. This is the text matrix to process using a shallow neural network (RNNs require a different format, as discussed later), which is always sized as `vocabulary_size` by the number of texts.

The numbers inside the matrix represent the number of times a word appears in the phrase. This isn't the only representation possible, though. Here are the others:

- **Frequency encoding:** Counts the number of word appearances in the phrase.
- **One-hot-encoding or binary encoding:** Notes the presence of a word in a phrase, no matter how many times it appears.
- **Term Frequency-Inverse Document Frequency (TF-IDF) score:** Encodes a measure relative to how many times a word appears in a document relative to the overall number of words in the matrix. (Words with higher scores are more distinctive; words with lower scores are less informative.)

Using the TF-IDF transformation instead

You can use the TF-IDF transformation from Keras directly. The `Tokenizer` offers a method, `texts_to_matrix`, that by default encodes your text and transforms it into a matrix in which the columns are your words, the rows are your texts, and the values are the word frequency within a text. If you apply the transformation by specifying `mode='tfidf'`, the transformation uses TF-IDF instead of word frequencies to fill the matrix values:

```
print(np.round(tokenizer.texts_to_matrix(texts,
                                         mode='tfidf'), 1))
```

The new output matrix looks like this:

```
[[0.  0.  0.7 0.7 0.9 0.9 0.9 0.9 0.  0.  0.  0.  0.  0.
```

```
  0. ]

 [0.  0.7 0.  0.  0.  0.  0.  0.  0.9 0.9 0.9 0.  0.  0.

  0. ]

 [0.  1.2 0.7 0.7 0.  0.  0.  0.  0.  0.  0.  0.9 0.9 0.9

  0.9]]
```

Note that by using a matrix representation, no matter whether you use binary, frequency, or the more sophisticated TF-IDF, you have lost any sense of word ordering that exists in the phrase. During processing, the words scatter in different columns, and the neural network can't guess the word order in a phrase. This lack of order is why you call it a bag-of-words approach. The bag-of-words approach is used in many machine learning algorithms, often with results ranging from good to fair, and you can apply it to a neural network using dense architecture layers. Transformations of words encoded into `n_grams` (discussed in the previous paragraph as an NLP processing transformation) provide some more information, but again, you can't relate the words.

Retaining order using RNNs

RNNs keep track of sequences, so they still use one-hot-encoding, but they don't encode the entire phrase; rather, they individually encode each token (which could be a word, a character, or even a bunch of characters). For this reason, they expect a sequence of indexes representing the phrase:

```
sequences = tokenizer.texts_to_sequences(texts)

print(sequences)
```

The output sequences look like this:

```
[[2, 3, 4, 5, 6, 7], [8, 9, 1, 10], [2, 3, 1, 11, 12, 13,
1, 14]]
```

TIP By matching the indexes found in the "Processing the text" section, earlier in this chapter, to the numbers in these lists, you can re-create the original sentences. For example, `2` is the `2: 'my'` in the index, which is the first word in `"My dog gets along with cats"`, the first entry in `texts`.

As each phrase passes to a neural network input as a sequence of index numbers, the number is turned into a one-hot encoded vector. You can use this code to see how the encoding works:

```
from keras.utils import to_categorical
print(to_categorical(sequences[0]))
```

The one-hot encoded vectors are then fed into the RNN's layers one at a time, making them easy to learn. For instance, here's the transformation of the first phrase in the matrix:

```
[[0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0.]]
```

In this representation, you get a distinct matrix for each piece of text. Each matrix represents the individual texts as distinct words using columns, but now the rows represent the word appearance order. (The first row is the first word, the second row is the second word, and so on.)

Using AI for sentiment analysis

Sentiment analysis computationally derives from a written text using the writer's attitude (whether positive, negative, or neutral), toward the text topic. This kind of analysis proves useful for people working in marketing and communication because it helps them understand what customers and consumers think of a product or service and thus act appropriately (for instance, by trying to recover unsatisfied customers or deciding to use a different sales strategy). Everyone performs sentiment analysis. For example, when reading text, people naturally try to determine the sentiment that moved the person who wrote it. However, when the number of texts to read and understand is too huge and the text constantly accumulates, as in social media and customer emails, automating the task is important.

The example in the following sections show a test run of RNNs using Keras and TensorFlow that builds a sentiment analysis algorithm capable of classifying the attitudes expressed in a film review. The data is a sample of the IMDb dataset that contains 50,000 reviews (split in half between train and test sets) of movies accompanied by a label expressing the sentiment of the review (0=negative, 1=positive). IMDb (`https://www.imdb.com/`) is a large online database containing information about films, TV series, and video games. Originally maintained by a fan base, it's now run by an Amazon subsidiary. On IMDb, people find the information they need about their favorite show as well as post their comments or write a review for other visitors to read.

Getting the IMDb data

Keras offers a downloadable wrapper for IMDb data. You prepare, shuffle, and arrange this data into a train and a test set. This dataset appears among other useful datasets at `https://keras.io/datasets/`. In particular, the IMDb textual data offered by Keras is cleansed of punctuation, normalized into lower-case, and transformed into numeric values. Each word is coded into a number representing its ranking in frequency. The most frequent words have low numbers; the less frequent words have higher numbers.

As a starting point, the code imports the `imdb` function from Keras and uses it to retrieve the data from the Internet (about a 17.5MB download). The parameters that the example uses encompass just the top 10,000 words, and Keras should shuffle the data using a specific random seed. (Knowing the seed enables you to reproduce the shuffle as needed.) The function returns two train and test sets, both made of text sequences and the sentiment outcome.

```
from keras.datasets import imdb

top_words = 10000
((x_train, y_train),
 (x_test, y_test)) = imdb.load_data(num_words=top_words,
                                    seed=21)
```

After the previous code completes, you can check the number of examples using the following code:

```
print("Training examples: %i" % len(x_train))
print("Test examples: %i" % len(x_test))
```

The following output shows that the examples are split evenly between training and testing:

```
Training examples: 25000
```

```
Test examples: 25000
```

This dataset is a relatively small one for a language problem; clearly the dataset is mainly for demonstration purposes. In addition, the code determines whether the dataset is balanced, which means it has an almost equal number of positive and negative sentiment examples.

```
import numpy as np
```

```
print(np.unique(y_train, return_counts=True))
```

Here's the output you should see:

```
(array([0, 1], dtype=int64), array([12500, 12500],
```

```
 dtype=int64))
```

The result, `array([12500, 12500])`, confirms that the dataset is split evenly between positive and negative outcomes. Such a balance between the response classes is exclusively because of the demonstrative nature of the dataset. In the real world, you seldom find balanced datasets.

Creating the review dictionaries

Now that you have a dataset to use, it's time to create some Python dictionaries that can convert between the code used in the dataset and the real words. In fact, the dataset used in this example is preprocessed and provides sequences of numbers representing the words, not the words themselves. (LSTM and GRU algorithms that you find in Keras expect sequences of numbers as numbers.)

```
word_to_id = {w:i+3 for w,i in
 imdb.get_word_index().items()}
id_to_word = {0:'<PAD>', 1:'<START>', 2:'<UNK>'}
id_to_word.update({i+3:w for w,i in
 imdb.get_word_index().items()})

def convert_to_text(sequence):
    return ' '.join(
       [id_to_word[s] for s in sequence if s>=3])

print(convert_to_text(x_train[8]))
```

Here's the output from this part of the example:

```
this movie was like a bad train wreck as horrible as it
 was you still had to continue to watch my boyfriend and i
 rented it and wasted two hours of our day now don't get
 me wrong the acting is good just the movie as a whole
 just both of us there wasn't anything positive or good
 about this scenario after this movie i had to go rent
 something else that was a little lighter jennifer is as
```

```
usual a very dramatic actress her character seems manic
and not all there hannah though over played she does a
wonderful job playing out the situation she is in more
than once i found myself yelling at the tv telling her to
fight back or to get violent all in all very violent
movie not for the faint of heart
```

The previous code snippet defines two conversion dictionaries (from words to numeric codes and vice versa) and a function that translates the dataset examples into readable text. As an example, the code prints the ninth example: "this movie was like a bad train wreck as horrible as it was ...". From this excerpt, you can easily anticipate that the sentiment for this movie isn't positive. Words such as *bad*, *wreck*, and *horrible* convey a strong negative feeling, and that makes guessing the correct sentiment easy.

TIP In this example, you receive the numeric sequences and turn them back into words, but the opposite is common. Usually, you get phrases made up of words and turn them into sequences of integers to feed to a layer of RNNs. Keras offers a specialized function, `Tokenizer` (see `https://keras.io/preprocessing/text/#tokenizer`), which can do that for you. It uses the methods `fit_on_text`, to learn how to map words to integers from training data, and `texts_to_matrix`, to transform text into a sequence.

However, in other reviews, you may not find revealing words like *bad*, *wreck*, and *horrible*. The feeling is expressed in a more subtle or indirect way, and understanding the sentiment early in the text may not

be possible because revealing phrases and words may appear much later in the discourse. For this reason, you also need to decide how much of the review you want to analyze. Conventionally, you take an initial part of the text, a *phrase,* and use it as representative of the entire review. Sometimes you just need a few initial words — for instance, the first 50 words — to get the sense; sometimes you need more. Especially long texts don't reveal their orientation early. It's therefore up to you to understand the type of text you're working with and decide how many words to analyze using deep learning. This example considers only the first 200 words, which should suffice.

Performing input padding

You may have noticed that the code starts encoding words beginning with the number 3, thus leaving codes from 0 to 2. Lower numbers are used for special tags, such as signaling the start of the phrase, filling empty spaces to have the sequence fixed at a certain length, and marking the words that are excluded because they're not frequent enough. This example picks up only the most frequent 10,000 words. Using tags to point out start, end, and notable situations is a trick that works with RNNs, especially for machine translation.

```
from keras.preprocessing.sequence import pad_sequences

max_pad = 200
x_train = pad_sequences(x_train,
                        maxlen=max_pad)

x_test = pad_sequences(x_test,
```

```
                        maxlen=max_pad)

print(x_train[0])
```

As output, you see the following list (shortened to fit in the book):

```
[  88    4 3310  406 6762    2    4  427 2140 1656…
   2  494   46 1954 4712  198   51   13  683 1193…
  89    4  114  495 7303  197    4 1168 1656   61…
  21   13  839   90  145    8  113   34 8253   27…
   6 8870 3310   88 8222   92    2    8 5388    5…
   2  449  168    6  404    2  112  207 1075    4…
 406 1522   13  124  903   97   90    2   21    2…
   2    2   93   61  492    2  305    7    2    4…
5679   83   27  117 2687 5419   29  941 1889   90…
 793    4 1526   84   37   28   34   96    7   49…
  56   23   61 2301 1111    9    4  255    8  937…
 159   29 1131   13 2134 3872   81   41   32   14…
 576 1301    5 5348 3134  255  335  170    8    2…
  29    9    2    2 3310  415   11 5215   89 1047…
 106   14   20  126]
```

By using the `pad_sequences` function from Keras with `max_pad` set to 200, the code takes the first two hundred words of each review. In case the review contains fewer than two hundred words, as many zero values as necessary precede the sequence to reach the required number of sequence elements. Cutting the sequences to a certain length and filling the voids with zero values is called *input padding*, an important processing activity when using RNNs like deep learning algorithms.

Designing an architecture

To perform an analysis of the reviews, you need to create a model that includes the needed algorithms. The resulting model is the architecture of your analysis engine. This example uses the following architecture:

```
from keras.models import Sequential
from keras.layers import Bidirectional, Dense, Dropout
from keras.layers import GlobalMaxPool1D, LSTM
from keras.layers.embeddings import Embedding

embedding_vector_length = 32
model = Sequential()
model.add(Embedding(top_words,
                    embedding_vector_length,
                    input_length=max_pad))
```

```
model.add(Bidirectional(LSTM(64, return_sequences=True)))

model.add(GlobalMaxPool1D())

model.add(Dense(16, activation="relu"))

model.add(Dense(1, activation="sigmoid"))


model.compile(loss='binary_crossentropy',

              optimizer='adam',

              metrics=['accuracy'])


print(model.summary())
```

The previous code snippet defines the shape of the deep learning model, where it uses a few specialized layers for natural language processing from Keras. The example also has required a summary of the model (`model.summary() command`) to determine what is happening with architecture by using different neural layers. Here's the summary of the model in this case:

```
_________________________________________________________________

Layer (type)                 Output Shape              Param #

=================================================================

embedding_1 (Embedding)      (None, 200, 32)           320000

_________________________________________________________________
```

```
bidirectional_1 (Bidirection (None, 200, 128)   49664

_________________________________________________________

global_max_pooling1d_1 (Glob (None, 128)        0

_________________________________________________________

dense_1 (Dense)              (None, 16)         2064

_________________________________________________________

dense_2 (Dense)              (None, 1)          17

=========================================================

Total params: 371,745

Trainable params: 371,745

Non-trainable params: 0

_________________________________________________________

None
```

You have the `Embedding` layer, which transforms the numeric sequences into a dense word embedding. A dense word embedding is easier for the layer of RNNs to learn, as discussed in the "Understanding Semantics Using Word Embeddings" section of Book 4, Chapter 4. Keras provides an `Embedding` layer, which, apart from having to be the first layer of the network, can accomplish two tasks:

- Applying pretrained word embedding (such as Word2vec or GloVe) to the sequence input. You just need to pass the matrix containing the embedding to its parameter `weights`.
- Creating a word embedding from scratch, based on the inputs it receives.

In this second case, `Embedding` needs to know:

- `input_dim`: The size of the vocabulary expected from data
- `output_dim`: The size of the embedding space that will be produced (the so-called dimensions)
- `input_length`: The sequence size to expect

After you determine the parameters, `Embedding` will find better weights to transform the sequences into a dense matrix during training. The dense matrix size is given by the length of sequences and the dimensionality of the embedding.

REMEMBER If you use The `Embedding` layer provided by Keras, you have to remember that the function provides only a weight matrix of the size of the vocabulary by the dimension of the desired embedding. It maps the words to the columns of the matrix and then tunes the matrix weights to the provided examples. This solution, although practical for nonstandard language problems, is not analogous to the word embeddings discussed previously, which are trained in a different way and on millions of examples.

The example uses `Bidirectional` wrapping — an LSTM layer of 64 cells. `Bidirectional` transforms a normal LSTM layer by doubling it: On the first side, it applies the normal sequence of inputs you provide; on the second, it passes the reverse of the sequence. You use this approach because sometimes you use words in a different order, and building a bidirectional layer will catch any word pattern, no matter the order. The Keras implementation is straightforward; you apply it as a function on the layer you want to render bidirectionally.

The bidirectional LSTM is set to return sequences (`return_sequences=True`); that is, for each cell, it returns the result provided after seeing each element of the sequence. The results, for

each sequence, is an output matrix of 200 x 128, where 200 is the number of sequence elements and 128 is the number of LSTM cells used in the layer. This technique prevents the RNN from taking the last result of each LSTM cell. Hints about the sentiment of the text could actually appear anywhere in the embedded words sequence.

In short, it's important not to take the last result of each cell, but rather the best result of it. The code therefore relies on the following layer, `GlobalMaxPool1D`, to check each sequence of results provided by each LSTM cell and retain only the maximum result. That should ensure that the example picks the strongest signal from each LSTM cell, which is hopefully specialized by its training to pick some meaningful signals.

After the neural signals are filtered, the example has a layer of 128 outputs, one for each LSTM cell. The code reduces and mixes the signals using a successive dense layer of 16 neurons with ReLU activation (thus making only positive signals pass through; see the "Choosing the right activation function" section of Book 4, Chapter 2 for details). The architecture ends with a final node using sigmoid activation, which will squeeze the results into the 0–1 range and make them look like probabilities.

Training and testing the network

Having defined the architecture, you can now train the network. Three epochs (passing the data three times through the network to have it learn the patterns) will suffice. The code uses batches of 256 reviews each time, which allows the network to see enough variety of words and sentiments each time before updating its weights using backpropagation. Finally, the code focuses on the results provided by the validation data (which isn't part of the training data). Getting a good result from the validation data means that the neural net is processing the input correctly. The code reports on validation data just after each epoch finishes.

```
history = model.fit(x_train, y_train,
                    validation_data=(x_test, y_test),
                    epochs=3, batch_size=256)
```

Getting the results takes a while, but if you are using a GPU, it will complete in the time you take to drink a cup of coffee. At this point, you can evaluate the results, again using the validation data. (The results shouldn't have any surprises or differences from what the code reported during training.)

```
loss, metric = model.evaluate(x_test, y_test, verbose=0)
print("Test accuracy: %0.3f" % metric)
```

The final accuracy, which is the percentage of correct answers from the deep neural network, will be a value of around 85–86 percent. The result will change slightly each time you run the experiment because of randomization when building your neural network. That's perfectly normal given the small size of the data you are working with. If you start with the right lucky weights, the learning will be easier in such a short training session.

In the end, your network is a sentiment analyzer that can guess the sentiment expressed in a movie review correctly about 85 percent of the time. Using even more training data and more sophisticated neural architectures, you can get results that are even more impressive. In marketing, a similar tool is used to automate many processes that require reading text and taking action. Again, you could couple a network like this with a neural network that listens to a voice and turns it into text. (This is another application of RNNs, which now power Alexa, Siri, Google Voice, and many other personal assistants.) The transition allows the application to understand the sentiment even in vocal expressions, such as a phone call from a customer.

Processing Time Series

It's possible to use RNNs for time series predictions. Unlike regression analysis, a time series prediction adds the complexity of a sequence to the prediction. Simply predicting a value overall is not enough; you must now predict that value based on where it fits in a sequence. For example, you may want to predict the sales for a store in a given month based on previous data for that month. Merely viewing the sales as a whole is not enough because stores commonly go through sales cycles such that 100 sales in January might be quite good, while 100 sales in July is abysmal. However, you also can't just use the sales for that month because the store will experience an overall sales trend that appears as sales increases or decreases in all the months as a whole.

Defining sequences of events

To better understand how a time series works, it pays to look at a dataset that includes a time series. The example in this and the sections that follow relies on the Airline Passengers Prediction dataset, found at `https://raw.githubusercontent.com/jbrownlee/Datasets/master/airline-passengers.csv`. The following code downloads the dataset when you don't already have it installed on your system:

```
import urllib.request

import os.path

filename = "airline-passengers.csv"

if not os.path.exists(filename):

    url = "https://raw.githubusercontent.com/\
```

```
jbrownlee/Datasets/master/airline-passengers.csv"

    urllib.request.urlretrieve(url, filename)
```

After you have a copy of the dataset, you can display the pertinent data as a plot so that you can see how the data varies over time using the following code:

```
%matplotlib inline

import pandas
import matplotlib.pyplot as plt

apDataset = pandas.read_csv('airline-passengers.csv',
                            usecols=[1])
plt.plot(apDataset)
plt.show()
```

Figure 5-1 shows how the data varies. Notice that the number of passengers varies over time, steadily increasing. However, the data also has a cycle that you must account for when working with it.

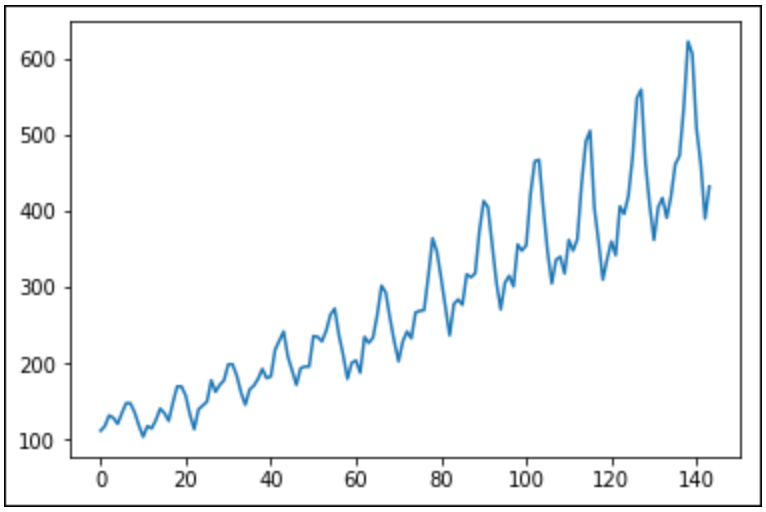

FIGURE 5-1: Working with cyclic data that varies over time.

Performing a prediction using LSTM

As mentioned in the "Considering the processing methodologies" section, early in this chapter, LSTM is one of the processing methodologies you have at your disposal when performing certain tasks. You can use it to perform predictions on time-series data using RNNs. The following sections show how to perform predictions on the Airline Passengers Prediction dataset using LSTM.

Creating training and testing datasets

The first step is to obtain the values from the dataset you just imported, using this code:

```
values = apDataset.values.astype('float32')
```

LSTM is sensitive to the range of data you provide. This next step normalizes the data so that it falls in the range between 0 and 1.

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler(feature_range=(0, 1))
```

```
normValues = scaler.fit_transform(values)
```

If you were to print `normValues`, you'd find that the values range between 0 and 1 as expected. As you can see from the graph in Figure 5-1, the dataset contains 144 entries, constituting 12 years of data. You need to split the normalized dataset into training and testing datasets. To provide enough training data, you want to use eight years of data for training (96 entries) and four years of data for testing (48 entries), using the following code:

```
train, test = normValues[0:96,:], normValues[96:144,:]

print("Train size: {0}, Test size: {1}".format(

    len(train), len(test)))
```

Managing the passenger sequence

The analysis you want to perform is all about sequence. You want to know how things happen in a time sequence, so the example uses special functions to create two datasets for training, `trainX` and `trainY`, where `trainX` is this month's passengers and `trainY` is next month's passengers. The same holds true for `testX` and `testY`. To better see how the analysis works, the following code performs the task in the original (before normalization) dataset first:

```
import numpy as np


np.random.seed(5)


def create_dataset(dataset):
```

```
        dataX, dataY = [], []
        for i in range(len(dataset)-2):
                a = dataset[i:(i+1), 0]
                dataX.append(a)
                dataY.append(dataset[i + 1, 0])
        return np.array(dataX), np.array(dataY)

valuesX, valuesY = create_dataset(values)

for a, b in zip(valuesX, valuesY):
    print("{0}   {1}".format(a, b))
```

The output (shortened for the book) shows how the next month, `b`, is always one ahead of this month, `a`:

```
[112.]   118.0
[118.]   132.0
[132.]   129.0
[129.]   121.0
[121.]   135.0
```

Of course, you now need to do the same thing to the normalized values, using this code:

```
trainX, trainY = create_dataset(train)

testX, testY = create_dataset(test)
```

Each row in the `trainX` and `testX` datasets are currently one-month samples. Within that sample is a feature — the number of passengers in a normalized form. So, a value like `[0.01544401]` is a sample feature configuration. To perform the analysis, what you really need is a sample, a step within that sample, and then a feature. The following code reshapes the `trainX` and `testX` datasets so that they appear as `[[0.01544401]]`:

```
trainX = np.reshape(trainX, (trainX.shape[0],
                             1, trainX.shape[1]))

testX = np.reshape(testX, (testX.shape[0], 1,
                           testX.shape[1]))
```

Defining the passenger analysis model

After all the required preparation is complete, you can finally create a model to perform the desired analysis. The following code specifies the elements in the model and then compiles it. It then fits the data to the model.

```
from keras.models import Sequential

from keras.layers import Dense

from keras.layers import LSTM

from sklearn.metrics import mean_squared_error

```

```
model = Sequential()

model.add(LSTM(4, input_shape=(1, look_back)))

model.add(Dense(1))

model.compile(loss='mean_squared_error', optimizer='adam')

model.fit(trainX, trainY, epochs=50, batch_size=1,

          verbose=2)
```

You could use additional epochs, but if you view the output, you see that the model stabilizes by epoch 25, so using 50 epochs is a little overkill.

Making a prediction

It's time to use the model to make a prediction. The following code makes a prediction using the training and testing models:

```
trainPredict = model.predict(trainX)

testPredict = model.predict(testX)
```

Because of the method used to perform the prediction, it's essential to invert the data so that you see the results in the original form of thousands of passengers per month:

```
trainPredict = scaler.inverse_transform(trainPredict)

trainY = scaler.inverse_transform([trainY])

testPredict = scaler.inverse_transform(testPredict)

testY = scaler.inverse_transform([testY])
```

Finally, you can calculate the root-mean-square-error (RMSE) of the predictions. This score shows the goodness of the model:

```
import math

trainScore = math.sqrt(mean_squared_error(
    trainY[0], trainPredict[:,0]))
print('Train Score: %.2f RMSE' % (trainScore))
testScore = math.sqrt(mean_squared_error(
    testY[0], testPredict[:,0]))
print('Test Score: %.2f RMSE' % (testScore))
```

Here's the output you can expect:

```
Train Score: 23.19 RMSE
Test Score: 49.74 RMSE
```

Table of contents collapsed